NeSy final report

Szymon Skwarek & Maciej Woś

June 2025

1 Introduction

We chose the task to reproduce experiments conducted in research paper "A Hybrid System for Systematic Generalization in Simple Arithmetic Problems" written by Flavio Petruzzellis, Alberto Testolin, and Alessandro Sperduti (link to the paper: https://arxiv.org/abs/2306.17249)

We succeeded in reproducing two experiments out of three which we suggested in the project proposal.

Our road to succeed in two experiments required several important steps. At the beginning we used default operations which are addition, substraction, multiplication, if statements, and for loops. After looking carefully into the pickled dataset, which is published in the original authors repository (https://github.com/flavio2018/nesy23/tree/main), we decided to reduce the dataset to the 3 basic operations which were included in the original training set. Those are addition, substraction, and multiplication. Moreover, we decided to use *step_generator*, which is the more advanced dataset generator, written in the original authors' repository.

In the next section, we define each of the experiments, present original paper's outcomes, and provide our obtained results in comparison to the original ones.

2 Experiment 1

In this experiment, we test the performance of different solver models with varying embedding dimensionality for different nesting values. We evaluate the performance with a use of both the sequence accuracy and the character accuracy.

2.1 Original Results

In this part we present the results obtained by the authors of the paper. They are shown in Figure 1. Their results show that a Transformer encoder-decoder using label positional encodings can generalize to out-of-distribution samples on the sub-expression solution task, independently of model size.

2.2 Our Results

The results (Table 1) we achieved are similar but not identical to the original ones. Some similarities can be seen, such as the character accuracy of small model for nesting 1 (53.6 ± 15.3) is close to original (57.2 ± 15.2) knowing that for the rest of the nesting numbers the accuracy is more than 90%. Similar pattern can be seen for other model sizes for both character and sequence accuracy, however there are some differences. In original results, the larger model, the better accuracy was for nesting of 1, but in our case usually medium model was the best for both character and sequence accuracy. In general trend, in original paper the results were very high for nesting number

	Nesting	1	2	3	4	5	6	7	8	9	10
Char Acc	small	57.2±17.5	92.1±15.2	99.2±1.3	97.6±4.9	98.4±4.5	99.5±1.2	99.9±0.2	99.3±0.9	98.1±2.8	98.9±1.6
	medium	72.2±17.4	96.8±5.4	99.7±1.0	100.0±0.0	100.0±0.0	99.7±0.8	100.0±0.0	100.0±0.1	99.5±1.5	99.6±1.2
	large	78.5±12.6	96.7±4.6	98.8±2.8	100.0±0.0	98.5±4.5	99.5±1.6	99.9±0.2	99.9±0.1	97.6±6.2	99.5±0.8
Seq Acc	small	28.1±25.4	82.3±27.8	98.5±2.6	93.5±12.9	97.1±8.0	99.2±2.1	99.4±1.5	98.4±1.9	96.2±6.1	97.4±3.5
	medium	50.8±27.4	93.6±11.0	99.1±2.7	99.9±0.3	100.0±0.0	99.2±1.8	100.0±0.0	99.8±0.6	99.3±1.6	99.2±2.4
	large	56.7±25.2	92.2±11.8	97.5±5.0	100.0±0.0	97.4±7.5	98.7±3.3	99.8±0.4	99.3±1.0	96.2±9.8	98.7±2.1

Figure 1: The original paper's results in the Experiment 1. The results are obtained on the sub-expression solution task. There are character and sequence accuracies of three different models: with 256-, 512-, and 1024- dimensional embeddings (those are respectively: small, medium, and large models). Authors report average and standard deviation of both metrics over 10 batches of 100 sequences. The figures show that the label positional embeddings endow the solver with strong out-of-distribution generalization capability on the sub-expression solution task.

Metric	Model	1	2	3	4	5	6	7	8	9	10
Char Acc	small	53.6±15.3	95.9±5.3	99.9±0.4	99.6±1.1	99.1±1.4	97.8±3.7	94.5±5.8	91.2±9.3	88.3±8.7	89.0±3.9
	medium	84.3±11.3	97.8±3.8	100.0±0.0	99.7±0.5	95.4±9.5	99.9±0.2	99.4±0.8	98.8±1.2	97.3±2.9	98.3±2.2
	large	70.8±12.6	93.0±15.1	98.3±3.7	99.9±0.2	99.5±1.1	99.3±1.4	99.4±1.3	100.0±0.1	99.8±0.3	98.8±2.1
Seq Acc	small	18.5±23.9	90.5±11.7	99.7±0.9	99.0±2.8	98.2±3.2	95.0±7.9	87.5±12.7	81.8±15.3	76.4±13.6	73.5±9.5
	medium	67.4±20.5	95.0±8.4	100.0±0.0	99.3±1.3	92.7±14.9	99.7±0.4	98.8±1.7	97.8±1.8	94.5±6.0	96.3±5.0
	large	45.1±20.3	87.4±26.0	96.8±6.6	99.8±0.5	99.0±2.1	98.7±2.9	98.9±1.9	99.9±0.2	99.5±0.9	97.6±4.1

Table 1: Our results in the Experiment 1. The results are obtained on the sub-expression solution task. There are character and sequence accuracies of three different models: with 256-, 512-, and 1024- dimensional embeddings (those are respectively: small, medium, and large models). We report average and standard deviation of both metrics over 10 batches of 100 sequences

higher than 2, but in replication study we noticed that the accuracy of small model in both character and sequence started to decrease after 5 or more nestings. Worth noticing is also, that often our standard deviation was smaller than original one.

3 Experiment 2

In second experiment we test the Transformer-based solver on the sub-expression solution task to evaluate its ability to generate correct simplification for innermost arithmetic operations. In the paper authors hypothesize that: "the solver module can leverage label positional encodings to achieve out-of-distribution generalization capacity on the sub-expression solution task". So to test that, during test time sovler receives complex arithmetical expressions with up to 10 nested operations, which is 8 more nested operations than seen during training. The goal remains to output the result of the innermost expression contained in the input.

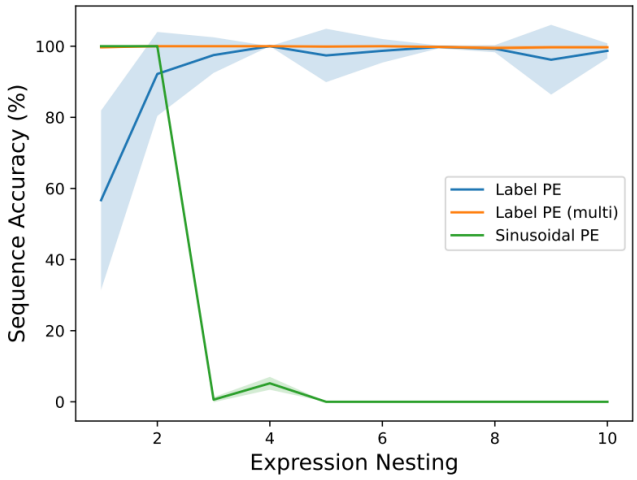

Figure 2: Original Results in the Experiment 2. Performance of solver modules trained with Label and Sinusoidal Positional Encodings. Label PE (multi) refers to a solver that generates 100 outputs per input, applying the same filtering rules as the combiner.

3.1 Original Results

The Figure 2 shows that the use of the use of Label Positional Encodings enables out-of-distribution generalization, which is not the case for Sinusoidal Positional Encodings.

3.2 Our Results

As shown in Figure 3, our results demonstrate similar trends for all positional encodings. In case of Label PE, the standard deviation is higher at expression nesting equal to 2, but then it remains stable. Lable PE (multi) differs the most from the original, as it starts from a lower sequence accuracy and has a higher standard deviation. At first look, Sinusoidal PE is also similar, but it should be noted that in the original experiment the sequence accuracy dropped to nearly zero at expression nesting equal to 3. In our case, the sequence accuracy dropped to around 20% at 4 nestings and decreased more steadily.

4 Experiment 3

The last experiment is about assessing the full system's performance on the arithmetic expression resolution task, where the solver and combiner work iteratively to simplify expressions with varying degrees of nested operations.

4.1 Original Results

Figure 4 shows that the combination of the architectural elements in their model allows to produce exact answers to harder problems than seen during training in a large fraction of cases. The difference between the Hybrid and Hybrid alt. methods is that the most frequently generated output for a given input is selected before filtering ill-formed outputs in the latter. The Figure 4 shows that the Hybrid alt. method is better than the baselines but worse than

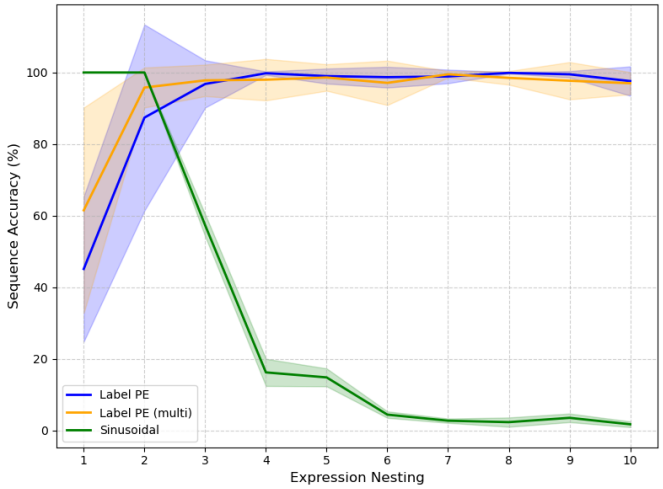

Figure 3: Our Results in the Experiment 2. Performance of different models, which use different Positional Encodings (Label or Sinusoidal), on sub-expression solution task. One model uses multi-output generation strategy in which we generate 100 outputs per input, applying the same filtering rules as the combiner.

Nesting	1	2	3	4	5	6	7	8	9	10
Character Accuracy	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0
Sequence Accuracy	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0
Avg Halting	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0	0.0±0.0

Table 2: Our Results in the Experiment 3 for multi-output generation strategy. Model is tested on the task of solution of the full expression. We report mean and standard deviation computed across 10 batches of 100 sequences per nesting value

the default combiner mechanism for harder problems due to the higher percentage of halted sequences. A halted sequence occurs when the solver's output does not comply with the required format. In this case, processing should stop because there is no meaningful way to continue simplifying. The similarity in sequence accuracy between the two methods for a smaller number of nested operations is due to the fact that the hybrid method produces more semantic errors in the final results. The Davinci model's performance shows that despite its large size and the amount of training data, the model cannot directly output solutions to complex arithmetic expressions. For an end-to-end trained Transformer, performance decreases with longer, more complex expressions but remains reasonably good with in-distribution samples.

4.2 Our Results

We did not manage to obtain any meaningful results in case when the multi-output generation strategy is employed. In this case we obtain only zeros for each metric as presented in Table 2. Note that in this case we tested only the Hybrid system. We did not test the baseline End-to-end model and the Davinci model, since we did not obtain any meaningful results for the Hybrid system.

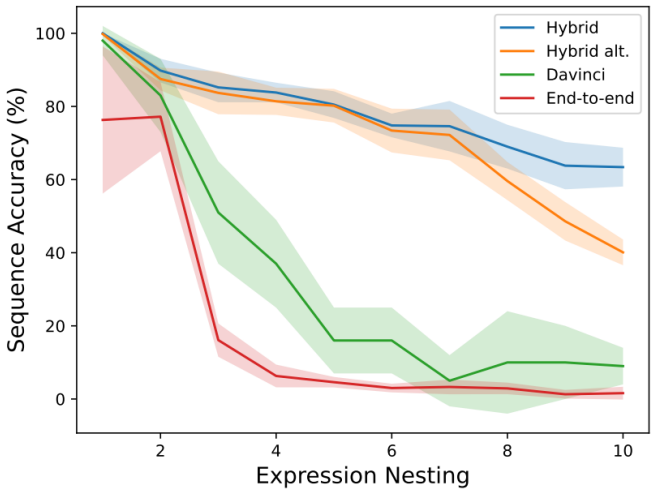

Figure 4: Original Results in the Experiment 3. Models are tested on the task of solution of the full expression. In all cases authors report mean and standard deviation computed across 10 batches of 100 sequences per nesting value, except for the Davinci model for which batches contain 10 sequences.